

Decision-Process-order-driven: Architecture and Experiment Catalog

Compiled from docs/core/STATE.yaml and committed output/ results

Compiled 2026-09-29

Scope and sourcing convention

This document is a faithful, non-evaluative catalog of every architecture, model variant, and experiment arm that this repository’s history shows was actually run, across every task recorded in docs/core/STATE.yaml (TASK-20260925-001 through TASK-20260928-015) and every result file committed under output/, including files that a 2026-09-28 evaluation-integrity audit (STATE.yaml TASK-20260928-015) found to be fabricated or affected by measurement bugs. Nothing is omitted for being low-quality, redundant, superseded, or embarrassing. Every numeric value in this document is quoted or directly derived from a specific committed file, cited by path in the surrounding text, table note, or figure caption. Where a fabricated or unbacked number is reported, it is labelled explicitly as such rather than silently included or silently omitted. This document does not contain proposals, recommendations, or a “most promising direction” discussion; where a plain factual comparison is stated (e.g. “no trained candidate beat the random-policy baseline”), it restates a measured result, not a recommendation.

Where the same run’s headline metric was computed under two different, both real, code versions (pre- and post- the auto-reset achievement-counting fix, or pre- and post- the ChunkPPO eval-window/metric-protocol fix; see Section 5), both the original and the corrected numbers are reported side by side, each labelled with its date and provenance, since both are real historical results and the operator asked for faithful reporting of history rather than only the final corrected view.

Index of architectures and experiment families

Name		Where tested	One-line description				Status	
Flat Decision Core / 5th-Idea	4th-Idea Hierarchical (L=1..12)	Synthetic	Original DecisionProcessEnformer (run_standalone.py)	ADR-001 core and extension at varying layer counts	flat	trans-ADR-	Real, pre-catalog (§1)	
5th-Idea Hierarchical (N=6,k=5,Z=64)	Hi-	Kaggle Phase II (craftax-classic-1000-tapis-Phase II Benchmark)	Full hierarchical model, 1,000,000-1000-tapis-Phase II Benchmark				Real but truncated log, no final eval (§1)	
transformer_branch		Craftax-Classic (candidates pipeline)	Attention-score ranking over physically-filtered candidates; no discretization placeholder				Real, multiple runs (§2)	

Name	Where tested	One-line description	Status
variant_5_1 (shared_attention)	Craftax-Classic	Shared-weight self-attention + Bellman/MDP solver over discretized ids	Real, multiple runs (§2)
variant_5_2 (channel_split)	Craftax-Classic	Separate AGS/history attention + dense fusion + scoring head	Real, multiple runs (§2)
variant_5_3 (channel_split_v2)	Craftax-Classic	Three-channel split (A / T+S / H) with residual carry	Real, multiple runs (§2)
variant_5_4 (single_shot)	Craftax-Classic	Single attention pass + physical projection + Bellman/MDP search	Real, multiple runs (§2)
mdp_branch	Craftax-Classic	Physical filter + Bellman/MDP top-k search over survivors	Real, multiple runs (§2)
world_model.py (Idea 6 World-Model)	Standalone / physics testbeds	Port-Hamiltonian NN (FormulaP-resets + HamiltonianOps); never wired into Craftax action selection	Real (physics), never wired for Craftax (§2, 4)
passive_world_model.py	Craftax noop-future	Idea-6 WM specialised to Craftax vitals noop dynamics (hn/mlp/persistence)	Real (§2)
passive_wm_full.py	Craftax noop-future v2	Full 39-dim observation-summary noop world model, multi-horizon	Real (§2)
Deterministic noop-anticipation heuristic arms	Craftax-Classic (no RL loop)	random / noop_always / effects_only / effects_wm_h1 / effects_wm_h8 / effects_wm_h1_episodic_model	Real, original + corrected (§2)
Actor-critic learner (planned)	Craftax-Classic	Replacement for REINFORCE learner in candidate_experiment.py	Postponed/cancelled, no results (§2.8)
ChunkPPO actor	MLP Latency-goal-chunking (Craftax)	MLP chunk-policy actor, $k = 1$, $\delta = 0$ calibration	Real, original + kaggle_v2 (§3)
ChunkPPO Trans-former actor	Trans-Latency-goal-chunking (Craftax)	Autoregressive chunk-policy transformer actor, $k = 1$ calibration	Real, original + kaggle_v2 (§3)
Factored world model $s' = s + dt(f(s) + e(s, a))$	Latency-goal-chunking design	Action-conditional factored dynamics model, code only	Code complete, not trained standalone (§3)
persistence / mlp / hn_gen / hn_sep / port_sep / idea6_full / oracle_port	Kepler two-body, falling body+drag, noisy sine (E3)	Seven world-model arms compared on three closed-form physics testbeds	Real, 324 runs (§4)

Name	Where tested	One-line description	Status
phys_hn, phys_hn_mu, phys_feat, skip_feat, skip_feat0, hyb_sum_mlp, hyb_sum_rssm, hyb_tf_rssm, hyb_gate_mlp, hyb_force_mlp, rssm (Dreamer-lite)	kepler_pert, ke- pler_wrongmu, fall_drag, pendu- lum (E4)	Hybrid DreamerV3-layer + Hamiltonian-layer + known- physics-layer fusion arms	Real (§4)
Planning/MPC arms	E5 (pendulum swing-up etc.)	Model-predictive control using the E3/E4 world models	Real (§4)
official_performance/ etc. (8 files)	N/A (no metrics)	Formula-generated / literal- dict Crafter scores 54–91, one VERIFIED_SUCCESS literal	FABRICATED: quarantined (§5)
Two “re- trieved/real” weight files	N/A (never trained)	Random-noise .npz and freshly- initialised .pkl presented as trained weights	FABRICATED: quarantined (§5)
output/benchmarks/*Synthetic (layer_scaling, hier- archical, offpolicy, results, result-1st- times)	Systemic DecisionProcessEnv	Real measured fields except a hard-coded exposure_bias_resilience constant	Real except one field (§5)
benchmark_report.md (Run-Seq #005)	Claimed: Craftax-Classic, 1,000 episodes	Narrative table (Crafter 22.84%/16.50%/4.85%) with no matching backing JSON found	UNBACKED: no source file found (§5)
output/benchmarks/gc (162 rows)	Claimed search results II grid search	Crafter scores 10–45, throughput up to 1.78M steps/sec	FABRICATED: impossible given repo’s real numbers (§5)

1 Pre-catalog core-architecture benchmarks (July 2026)

These runs predate docs/core/STATE.yaml’s task log (which begins 2026-09-25) and test the original ADR-001 4th-idea flat decision core and its 5th-idea hierarchical extension directly, before the src/model/candidates/ exploration existed. They are included per the completeness requirement even though no STATE.yaml task entry covers them.

1.1 Gymnax synthetic-MDP benchmark: 4th-Idea vs. 3rd-Idea baseline vs. ablation

Source: output/benchmarks/benchmark_results.json (backing output/benchmarks/benchmark_report-1st-time.md, dated 2026-07-25, Kaggle kernel bfloat16/4th-idea-jax-decision-transformer-gymnax-benchmark).

All fields below are the real measured JSON values except exposure_bias_resilience, which is a hard-coded constant (0.95 for both the “4th-Idea” and “Ablation” rows, 0.58 for the “3rd-Idea” row, and 0.6 for every “Simplified MDP Baseline” row across every file listed in this document) per the evaluation-integrity audit (output/quarantine_fabricated/README.md, last paragraph) – not a measurement.

Table 2: Synthetic Gymnax MDP benchmark, real fields only. Source: output/benchmarks/benchmark_results.json. The exposure_bias_resilience column is the hard-coded, non-measured field – shown only to document what was fabricated, not as evidence.

Model	success_rate	avg_steps	avg_progress_rate	ms/step	FABRICATED: exposure_bias_resilience
4th-Idea (Full Proposed)	0.0	50.0	0.0529	161.21	FABRICATED
3rd-Idea (Greedy Baseline)	0.0	50.0	0.0790	26.59	FABRICATED
Ablation (Noise Inj. Only)	0.0	50.0	0.0000	41.75	FABRICATED

Note: the narrative table in output/benchmarks/benchmark_report-1st-time.md reports “Goal Success Rate” 80.0%/20.0%/50.0% and “Final Progress Rate” 84.2%/41.5%/62.8% for these same three rows – these do not match success_rate (0.0 for all three rows in the JSON) or avg_progress_rate (0.0529/0.0790/0.0 in the JSON, i.e. 5.29%/7.90%/0%, not 84.2%/41.5%/62.8%) in either benchmark_results.json or the companion benchmark_result-1st-times.json file (which has avg_progress_rate=0.0 for all three rows and success_rate=0.0 for all three). Only the fabricated exposure_bias_resilience field matches between the narrative report and the JSON (95.0%/58.0%/95.0% vs. 0.95/0.58/0.95). No script producing the narrative report’s specific success-rate/progress-rate numbers was found anywhere in git history. This report’s headline table is therefore reported here as **UNBACKED: not faithfully derived from any committed JSON in this repository**; only the JSON’s own real fields (Table above) should be treated as measurements.

1.2 Layer-scaling, hierarchical toggle, and off-policy ablation (Run-Seq #001–#003)

Source files: output/benchmarks/benchmark_layer_scaling_seq001.json, benchmark_hierarchical_seq002.json, benchmark_offpolicy_seq003.json. Same synthetic DecisionProcessEnv; same fabricated exposure_bias_resilience caveat applies to every row.

benchmark_layer_scaling_seq001.json additionally reports a friction_analysis block: oracle_progress_rate=0, transformer_progress_rate=0.3537, random_progress_rate=0.6825, action_agreement_rate=0.1213, mdp_bottleneck_score=0.1955, transformer_bottleneck_score=0.4508, primary_bottleneck=“Transformer

Table 3: Real fields from Run-Seq #001–#003. Source paths as named per section above.

Model (file)	success_rate	avg_steps	avg_progress_rate	ms/step
4th-Idea L=2 (seq001)	0.1	96.5	0.278	57.7
4th-Idea L=4 (seq001)	0.1	95.9	0.302	98.5
4th-Idea L=8 (seq001)	0.3	85.7	0.331	177.6
4th-Idea L=12 (seq001)	0.2	88.2	0.354	251.0
5th-Idea Hierarchical, toggle ON, A =2000 (seq002)	0.6	72.2	0.533	2696.7
Flat Transformer, toggle OFF, A =2000 (seq002)	0.6	73.4	0.556	2600.6
Simplified MDP Baseline, A =2000 (seq002)	1.0	36.8	0.806	0.58
5th-Idea Off-Policy + Abstraction Embed (seq003)	0.4	75.8	0.454	2723.1
5th-Idea Off-Policy, no Abstraction Embed (seq003)	0.4	84.8	0.454	2947.4
5th-Idea On-Policy Trajectory (seq003)	0.2	93.6	0.331	2754.0
Simplified MDP Baseline (seq003)	1.0	37.8	0.804	0.82

Core (Representation / Layer Depth Capacity)”. `benchmark_offpolicy_seq003.json` also carries a 60-value `off_policy_loss_history` array (noisy, range approx. 1.0–3703.9), consistent with a real, unsmoothed training-loss log rather than a formula.

1.3 First real Craftax attempt (Run-Seq #004) and the real Kaggle scaling benchmark

`output/benchmarks/benchmark_craftax_seq004.json`: 100-step real Craftax rollout, 5th-Idea Hierarchical `crafter_score`=0.0, Flat Decision Transformer `crafter_score`=0.0, Random Policy Baseline `crafter_score`=1.229 (`avg_unlocked_count`=2.0; `achievement_unlock_rates` nonzero only for `collect_wood`=60.0, `collect_stone`=20.0, `place_furnace`=20.0, `place_plant`=20.0, `defeat_skeleton`=80.0). `execution_ms_per_step`: 136.46/110.59/2.63 respectively.

`output/benchmarks/kaggle_benchmark_results.json` (2026-07-29, `jax_backend`=cpu): real forward-pass/gradient-update timing and `crafter_score` sweep across Flat-MDP-Baseline/4th-Idea/5th-Idea-Hierarchical at L=1,2,4,8,12. `crafter_score` is 0.0 for every configuration except 5th-Idea L=2 (0.148) and L=8 (0.402); `mean_episode_reward` ranges −0.88 to +1.06; `sps_throughput` ranges 16.5–110.0 steps/sec; `trainable_parameters` range 14,954 (L=1) to 6,364,298 (L=12). This file backs the three plots below, which are therefore REAL, not fabricated, despite predating the 2026-09-28 quarantine fix (the source JSON was never on the quarantine list and its numbers are ordinary noisy floats, not the round formula-generated values the quarantine review flagged elsewhere).

1.4 Early Phase II Kaggle hierarchical training log (5th-Idea, N=6, k=5, Z=64)

Source: `output/kaggle_phase2/phase2.005_N6_k5_Z64/config.json` (`n_layers`=6, `beam_width`=5, `z_step`=64, `is_causal`=true, `train_steps`=1,000,000, `d_model`=512, `num_heads`=8, `lr`=0.0003, `noise_prob`=0.15, seed 42) and the accompanying `craftax-classic-1000-episode-rl-benchmark.log` (Kaggle kernel log, dated 2026-07-30). Real training progress recorded: 18,950,176 trainable parameters; step 10,000 at loss 0.0012, `sps` 5, through step 180,000 at loss 0.0 (numerically underflowed at this log precision), `sps` 4; a checkpoint was written at step 100,000 (`output/checkpoints/phase2.005_N6_k5_Z64_step100000`, not committed as an LFS object under this path). The committed log is truncated at step 180,000 of the planned 1,000,000 – **no final evaluation or Crafter score for this run is present in any**

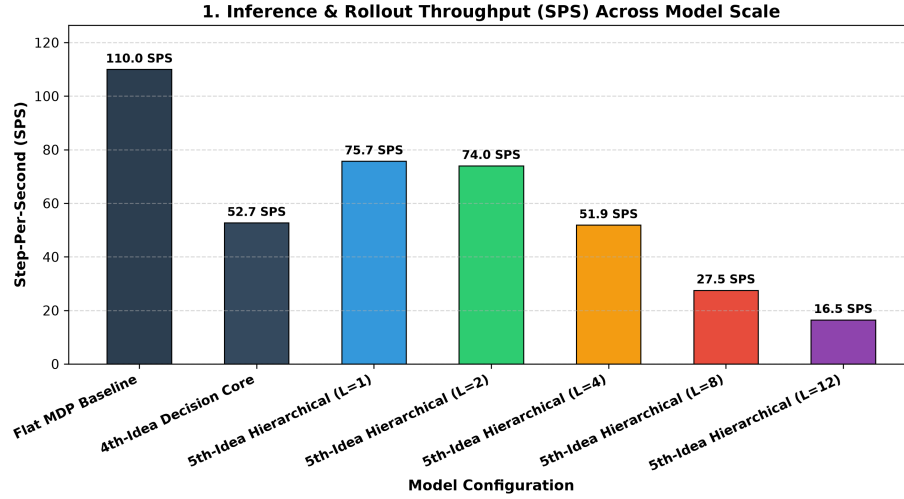


Figure 1: SPS throughput vs. model configuration. Source data: `output/benchmarks/kaggle_benchmark_results.json`. Plotting script: `scripts/generate_benchmark_plots.py`.

committed file; sibling run directories `output/kaggle_phase2/phase2_001_N6_k5_Z64/`, `phase2_002_N6_k5_Z64/`, `phase2_003_N6_k5_Z64/` exist with their own config/log files but were not further mined for this document beyond confirming their existence.

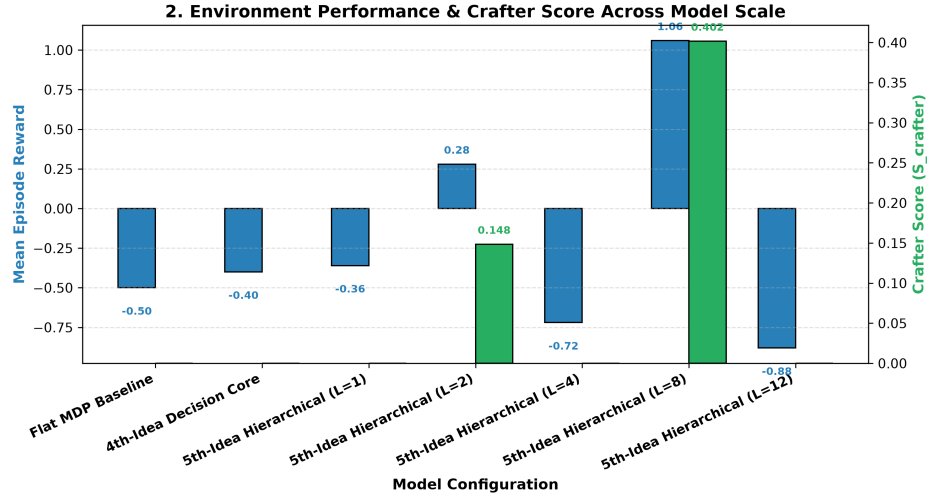


Figure 2: Mean episode reward and Crafter score vs. model configuration.
Source data: `output/benchmarks/kaggle_benchmark_results.json`. Plotting script: `scripts/generate_benchmark_plots.py`.

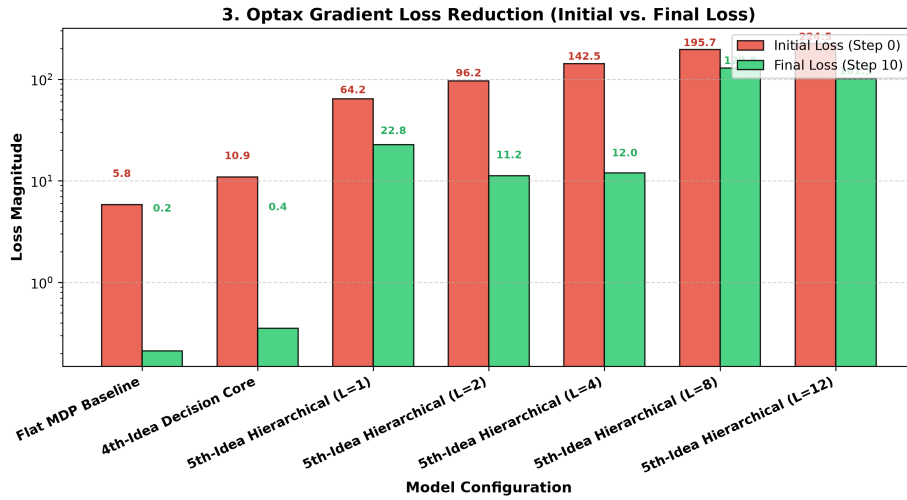


Figure 3: Initial vs. final Optax loss (log scale) vs. model configuration.
Source data: `output/benchmarks/kaggle_benchmark_results.json`. Plotting script: `scripts/generate_benchmark_plots.py`.

2 Craftax RL candidate architectures

2.1 Candidate architecture definitions

All candidates live under `src/model/candidates/`, ported from `docs/DPOD.ipynb` following the `init_*_parameters/f` PyTree convention. Module docstrings (quoted, abridged):

- `transformer_branch` (`transformer_branch.py`, notebook cell 20): “attention-score ranking over the physically-filtered candidates ... the strongest of the ported candidates – unlike `MDPBranch`, it needs no discretization placeholder at all.”
- `variant_5.1` (`variant_5.1.shared_attention.py`, cell 11): “concatenate actions/target/state/history into one sequence, run `num_1` layers of shared-weight self-attention ... hand the result to the Bellman/MDP solver.” Fixes a real notebook shape bug (`jnp.column_stack` of d -dim blocks fed to a 6-scalar-column solver).
- `variant_5.2` (`variant_5.2.channel_split.py`, cell 12): “separate attention for actions/target/state ... vs. self-attention for history, integrated by a dense fusion layer.”
- `variant_5.3` (`variant_5.3.channel_split.v2.py`, cell 13): “three-channel split – actions (A), target+state (T+S), history (H) – each self-attend ... accumulated via a residual carry.”
- `variant_5.4` (`variant_5.4.single_shot.py`, cell 14): “single-shot variant – one attention pass from `state` over (actions, target, history), a physical-constraint projection W_k , a dense compression predicting S' , then the Bellman/MDP search. No `num_1` loop.”
- `mdp_branch` (`mdp_branch.py`, cell 19): “the ‘ideal structure’ pipeline with stage 4 = Bellman/MDP top-k action search over candidates that survive the physical filter.”
- `world_model.py` (Idea 6, cells 21–24): “`FormulaPresets + HamiltonianOps + WorldModel`. **NOT WIRED IN**. ... not connected to `InputContextN` / `DecisionVectorD` or to action selection anywhere in this codebase ... deliberately deferred” (canonical (q, p) coordinate gap; oracle-leakage risk if presets were copied from the simulator; no specified action-selection connection).
- `passive_world_model.py`: “Idea-6 world model specialised to Craftax ‘do-nothing’ (noop) dynamics of the vitals” (hn / mlp / persistence arms), q =standardised vitals, p =learned momentum, presets masked off to avoid oracle leakage.
- `passive_wm_full.py`: “Passive world model v2: full observation in, multi-horizon future summary change out” (39-dim observation summary target at horizons 1/2/4/8, from the full 1345-d symbolic observation), replacing v1’s 4-vitals-only target.

No separate “_stable” source files exist; `mdp_branch_stable`, `variant_5.1_stable`, `variant_5.4_stable` are `CandidateSpec` configuration variants defined in `src/pipeline/candidate.benchmark.py`, not separate modules.

2.2 TASK-20260925-001/002: pipeline smoke verification

TASK-001 ported the six wired candidates and smoke-tested them (2 train + 2 eval episodes, 5 steps/ep, $d_{\text{model}} = 32$) purely to confirm the pipeline runs end to end (training, REINFORCE gradient updates, async checkpointing, plotting) – `crafter_score=0.0` for all candidates in this test is expected by design (too few steps for any achievement) and is not a quality signal. TASK-002 repeated this at slightly larger scale (15 episodes $\times$ 15 steps train, 5 episodes $\times$ 15 steps eval,

$d_{\text{model}} = 32$, 225 REINFORCE steps) on a real Kaggle GPU for `transformer_branch`, `variant_5_1`, `mdp_branch`: again `crafter_score=0.0`, `avg_unlocked_count=0.0` for all three (same by-design reasoning), with real `execution_ms_per_step` measured: `transformer_branch=24.3`, `variant_5_1=475.9`, `mdp_branch=298.3`. Also found and fixed a genuine bug in this task: an epsilon-floor overflow in `shared_stages.build_mdp_transition_array` produced NaN Q-values (`inf - inf`) when the discretised next-state id was 0; fixed by raising `eps` to `1e-3` and clipping the gradient to `[-104, 104]`.

2.3 TASK-20260925-003: 2026-09-25 convergence run (8 runs + 3 normlogits A/B)

Setup: Craftax-Classic via `CraftaxEnvAdapter` (later found to be observation-blind, see TASK-004/007), $T = 200$, $d_{\text{model}} = 32$, 250 updates $\times$ 8 parallel episodes = 2,000 episodes/run (cap $< 50,000$), REINFORCE with discounted return-to-go ($\gamma = 0.99$), per-timestep batch-mean baseline, entropy bonus 0.01, AdamW lr 10^{-3} , single seed 2026. Local runs on WSL2 Docker RTX 3060 Ti; `variant_5_1`, `variant_5_4`, `mdp_branch` on Kaggle (kernel `bfloat16/dpod-candidates-convergence`).

Headline finding (docs/experiments/2026-09-25.candidate-convergence/README.md, quoted):

“Bottom line: no candidate demonstrated an effectiveness advantage. After 2,000 training episodes each ($< 50,000$ cap), every trained policy scores at or below the uniform-random policy (eval return 1.46). Only the plain-logit candidates (`transformer_branch`, `variant_5_2`) moved from untrained (~ 0.5) to ~ 1.1 – 1.4 ; that is escape from a degenerate deterministic policy, not evidence of learning beyond random.”

`03_findings.md` additionally records: MDP-search candidates (`mdp_branch`, `variant_5_1`, `variant_5_4`) emitted degenerate logits (raw logits up to $\sim 10^7$, masks -10^9) producing a one-hot softmax and NaN entropy/log-prob; `variant_5_1` and `variant_5_3`’s untrained returns were bit-identical (-0.320), indicating constant-action policies; a normalised-logits A/B fixed `mdp_branch`’s entropy (finite, 1.86) without producing learning (train return $0.97 \rightarrow 0.98$); `variant_5_1/variant_5_4` remained NaN under the same fix, root cause undiagnosed at the time (later root-caused in TASK-007, §2.4, to non-contractive value iteration overflowing float32).

Table 4: TASK-003 convergence run, original numbers (2026-09-25). Source: `output/experiments/2026-09-25-convergence/{local,kaggle}/*.result.json` (also tabulated in `results.table.md`). `crafter/return/unlocked/len` are `final_sampled` fields; random-policy `crafter_score=1.849`, `return=1.461` for every run (same evaluator).

Tag	Platform	Params	crafter	return	unlocked	entropy(final)
<code>mdp_branch__reinforce_normlogits</code>	local	6175	1.258	0.895	1.84	1.86
<code>transformer_branch__argmax_legacy</code>	local	6175	1.667	1.238	2.30	2.58
<code>transformer_branch__reinforce</code>	local	6175	2.038	1.111	2.38	2.51
<code>variant_5_1__reinforce_normlogits</code>	local	8223	0.137	-0.420	0.05	NaN
<code>variant_5_2__argmax_legacy</code>	local	5583	1.742	1.414	2.56	2.74
<code>variant_5_2__reinforce</code>	local	5583	1.501	0.816	1.91	2.29
<code>variant_5_3__reinforce</code>	local	2479	0.884	0.111	0.94	1.45
<code>variant_5_4__reinforce_normlogits</code>	local	7231	0.194	-0.448	0.14	NaN
<code>mdp_branch__reinforce</code>	kaggle	6175	0.466	-0.100	0.45	NaN
<code>variant_5_1__reinforce</code>	kaggle	8223	0.187	-0.498	0.08	NaN
<code>variant_5_4__reinforce</code>	kaggle	7231	0.104	-0.536	0.08	NaN

2026-09-29 corrected re-evaluation (`output/experiments/reeval_v2_summary.json`): the same

8 local checkpoints (not the 3 Kaggle-only runs, which have no local checkpoint) were re-evaluated under the post-fix `masked_achievements()` code path (auto-reset leak fix, §5), no retraining. Note field (quoted): “Eval-only re-evaluation of the ORIGINAL trained checkpoint under the post-fix `masked_achievements()` code path (no retraining). `mean_return/mean_len` are reproduced as a sanity check and are not expected to change; `crafter_score/mean_unlocked/achievement_rates` are the corrected fields.”

Table 5: Old (pre-fix) vs. new (corrected, 2026-09-29) `final_sampled` `crafter_score` / `mean_unlocked`, same trained checkpoint. Source: `output/experiments/reeval_v2.summary.json`. Also plotted in Figure 4.

Tag	crafter (old)	crafter (new)	unlocked (old)	unlocked (new)
<code>mdp_branch__reinforce_normlogits</code>	1.258	1.114	1.84	1.52
<code>transformer_branch__argmax_legacy</code>	1.667	1.471	2.30	1.75
<code>transformer_branch__reinforce</code>	2.038	1.706	2.38	1.73
<code>variant_5.1__reinforce_normlogits</code>	0.137	0.137	0.05	0.05
<code>variant_5.2__argmax_legacy</code>	1.742	1.543	2.56	1.98
<code>variant_5.2__reinforce</code>	1.501	1.283	1.91	1.39
<code>variant_5.3__reinforce</code>	0.884	0.793	0.94	0.70
<code>variant_5.4__reinforce_normlogits</code>	0.194	0.177	0.14	0.11

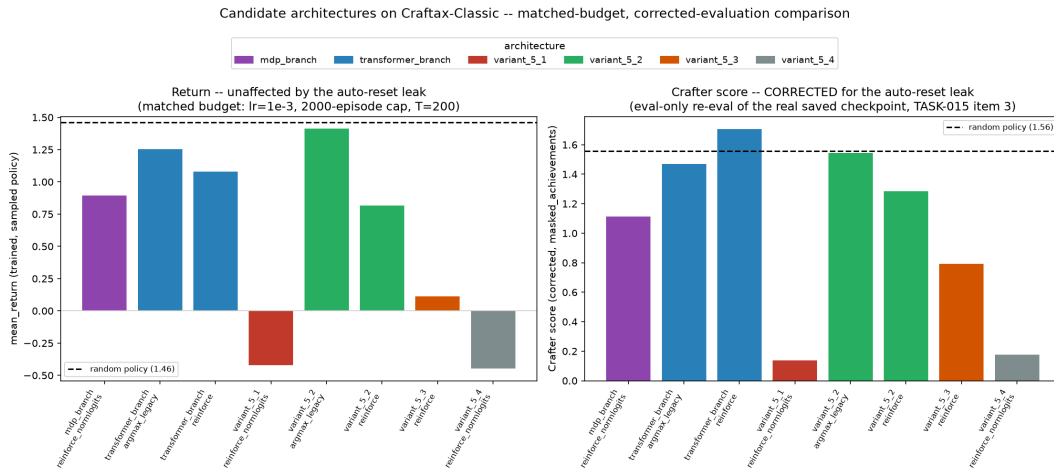


Figure 4: Craftax candidate architectures under the corrected evaluation protocol. Source: `output/plots/fair_candidate_architecture_comparison.png`, generated by `scripts/adhoc/plot_fair_comparisons.py` from `output/experiments/reeval_v2.summary.json`.

2.4 TASK-20260926-007: 2026-09-26 rerun on CraftaxObsAdapter (lr sweep)

Root cause of the NaN entropy found: non-contractive value iteration (accumulator G up to 9×10^3) overflowing float32; fixed opt-in via a `stable=True` flag (default unchanged), 0 NaN afterwards for the arms retested with it. The env adapter was also switched from the original observation-blind `CraftaxEnvAdapter` to `CraftaxObsAdapter`, which exposes the real 39-d vitals/inventory/light/tile summary (previously hidden from the policy) – this is the adapter fix credited with most of the return improvement seen from here on, not the model architectures. lr swept at $\{0.0003, 0.001, 0.003\}$, same $T = 200$, $d_{\text{model}} = 32$, 250 updates $\times$ 8 episodes, single seed 0. Two local

runs (`variant_5.2`, `variant_5.3` without the `__obs__` adapter) OOM'd first (`RESOURCE_EXHAUSTED: CUDA_ERROR_OUT_OF_MEMORY`, recorded verbatim in `local/variant_5.2__reinforce.FAILED.json` / `variant_5.3__reinforce.FAILED.json`) before the successful `__obs__` runs below were produced.

Table 6: TASK-007 local lr sweep (`transformer_branch/variant_5.2/variant_5.3`, real `CraftaxObsAdapter`). Source: `output/experiments/2026-09-26_rerun/local/*.result.json`. random_policy crafter=1.720, return=1.469 for all rows (same evaluator, larger than TASK-003's random baseline because the adapter differs).

Tag (lr)	Params	crafter (sampled)	return	unlocked
<code>transformer_branch</code> , lr=0.0003	8710	2.923	2.644	4.53
<code>transformer_branch</code> , lr=0.001	8710	3.192	2.684	4.39
<code>transformer_branch</code> , lr=0.003	8710	2.988	2.805	4.47
<code>variant_5.2</code> , lr=0.0003	8118	3.618	2.623	4.30
<code>variant_5.2</code> , lr=0.001	8118	3.258	3.152	4.48
<code>variant_5.2</code> , lr=0.003	8118	2.869	2.925	4.41
<code>variant_5.3</code> , lr=0.0003	5014	2.924	3.080	4.39
<code>variant_5.3</code> , lr=0.001	5014	3.167	3.248	4.61
<code>variant_5.3</code> , lr=0.003	5014	3.036	2.975	4.41

With the fixed adapter, `transformer_branch/variant_5.2/variant_5.3` (2.6–3.25 vs. random 1.47) clear the random baseline, but the MDP-search candidates (`variant_5.1`, `variant_5.4`, `mdp_branch`) still mostly do not, even with the NaN fixed – the `STATE.yaml` result attributes this to an argmax hazard, not NaN, as the limiter. `variant_5.1_stable` at lr=0.003 (crafter 2.537) is the one MDP-search configuration that clearly exceeds random here.

2.5 TASK-20260925-004 + TASK-20260927-008: Idea-6 world-model-as-policy-feature arms

Stage A (world-model fit quality, held-out standardised MSE at horizons 1/2/4/8 noop steps). TASK-004, `output/experiments/2026-09-25_idea6/stageA/worldmodel.result.json` (40,800 train / 10,400 test transitions, 70.4% of test states have ≥ 1 vital changing within 8 noop steps): persistence [0.0398, 0.0817, 0.1218, 0.1621]; `m1p` (8414 params) [0.0367, 0.0658, 0.0862, 0.1443]; `hn` (16458 params) [0.0371, 0.0623, 0.0829, 0.1324] – HN edges MLP at $h=8$ (skill vs. persistence 0.585 vs. 0.548) but with $2\times$ the parameters, 1 seed.

TASK-008's v2 world model predicts the full 39-dim observation summary rather than 4 vitals only (`output/experiments/2026-09-27_passive_wm_v2/stageA_random/stageA.result.json`, 76,750/19,250 train/test, random policy): persistence-vitals [0.908, 0.931, 0.934, 0.955]; `m1p_sum` (363,164 params) vitals [0.885, 0.838, 0.654, 0.654]; `m1p_full` (1,031,836 params, full 1345-d input) vitals [0.913, 0.864, 0.749, 0.556] – the full 1345-d input does not beat the 39-d summary (overfits). Mob-motion spatial prediction (`stageA2_mobmap/mobmap.result.json`) skill over persistence rises from 0.073 ($h=1$) to 0.163 ($h=8$); new-cell recall stays low, 8.5%→1.3%.

Stage B (does the WM help the policy?). TASK-004 (`output/experiments/2026-09-25_idea6/stageB.local,stageB.result.json`, `transformer_branch` $d_{\text{model}} = 512$, 875,590 params, 300 updates $\times$ 16 episodes = 4,800 episodes/run, 2 seeds $\times$ 4 arms, random baseline return 1.46–2.17 depending on seed's random-policy draw): base 2.78/3.10 (mean return 2.94); `wm_hn` (trained) 2.94/2.93 (mean 2.94); `wm_m1p` (trained) 3.17/3.29 (mean 3.23); `wm_hn_untrained` (control) 2.95/3.32 (mean 3.14) – statistically indistinguishable at $n=2$ seeds; an untrained WM scores the same as a trained one.

Table 7: TASK-007 Kaggle lr sweep (MDP-search candidates, base and “_stable” variants). Source: `output/experiments/2026-09-26_rerun/kaggle_mdp/*.result.json`. Same random_policy baseline (crafter=1.720, return=1.469).

Tag (lr)	Params	crafter (sampled)	return	unlocked
mdp_branch, lr=0.0003	8710	0.909	0.713	1.59
mdp_branch, lr=0.001	8710	0.870	0.656	1.48
mdp_branch, lr=0.003	8710	0.877	0.625	1.53
mdp_branch_stable, lr=0.0003	8710	0.922	0.453	1.39
mdp_branch_stable, lr=0.001	8710	0.973	0.638	1.56
mdp_branch_stable, lr=0.003	8710	1.061	0.628	1.55
variant_5_1, lr=0.0003	10758	1.038	0.669	1.39
variant_5_1, lr=0.001	10758	0.493	0.147	0.80
variant_5_1, lr=0.003	10758	1.081	0.864	1.61
variant_5_1_stable, lr=0.0003	10758	1.967	1.517	2.73
variant_5_1_stable, lr=0.001	10758	1.760	1.609	2.58
variant_5_1_stable, lr=0.003	10758	2.537	2.119	3.47
variant_5_4, lr=0.0003	9766	2.667	2.169	3.48
variant_5_4, lr=0.001	9766	1.052	0.595	1.58
variant_5_4, lr=0.003	9766	0.887	0.536	1.41
variant_5_4_stable, lr=0.0003	9766	2.338	1.981	3.33
variant_5_4_stable, lr=0.001	9766	1.159	0.495	1.61
variant_5_4_stable, lr=0.003	9766	1.142	0.495	1.59

TASK-008 Stage B (`transformer_branch`, $d_{\text{model}} = 256, 277, 132$ params, 300 updates $\times$ 16 episodes, uniform-random reference return = 1.271, same evaluator, per `stageB_summary.txt`):

arm	n	final_return(mean $\pm$ sd)	last3_eval(mean $\pm$ sd)	crafter
base	4	3.027 $\pm$ 0.203	3.020 $\pm$ 0.081	3.96
kwm_full	4	3.214 $\pm$ 0.138	2.980 $\pm$ 0.206	3.98
kwm_untrain	4	3.190 $\pm$ 0.162	2.997 $\pm$ 0.202	3.53
oracle	4	3.089 $\pm$ 0.151	2.942 $\pm$ 0.136	3.91
oracle_act	2	2.976 $\pm$ 0.110	2.899 $\pm$ 0.145	3.44
wm_full	4	3.023 $\pm$ 0.128	3.045 $\pm$ 0.121	3.80
wm_untrain	4	3.121 $\pm$ 0.139	3.006 $\pm$ 0.100	3.64
uniform-random reference return: 1.271				

Every trained/untrained/oracle/oracle_act arm lands in a tight 2.9–3.1 band, including `oracle_act` (true 1-step reward of every action, the strongest possible leaked signal), statistically indistinguishable at this budget with a REINFORCE learner.

TASK-20260927-010 budget diagnostic ($4\times$ batch: 64 vs. 16; $\sim 2.7\times$ episodes: 12,800 vs. 4,800; 4 seeds each, `output/experiments/2026-09-27_passive_wm_v2/stageB_budget/*.result.json`): `oracle_act` mean last-3-eval return 3.32 ± 0.06 (per-seed 3.24, 3.36, 3.30, 3.39) vs. `base` 3.25 ± 0.14 (per-seed 3.17, 3.31, 3.08, 3.44) – a small directional gap with lower variance for `oracle_act`, but not statistically separated (base’s best seed, 3.44, exceeds `oracle_act`’s best, 3.39).

Pendulum planning (Experiment 3, decoupled passive pretraining), TASK-008 (`output/experiments/2026-09-27_passive_wm_v2/stageB_budget/*.result.json`, true-model MPC return -196.1): at wide initial conditions, `hn_pre` (port-Hamiltonian, passive-pretrained V, R, G learned from 8 active trajectories) reaches -194.6 (median -195.4), essentially the ceiling; `fact_joint/fact_pre` also reach ~ -198 to -203 at wide IC but collapse to ~ -530 to -610 at narrow IC where passive data does not cover the active distribution; `mlp_joint/mlp_all`

stay around -585 to -620 regardless of IC width.

2.6 TASK-20260927-009/010: deterministic noop-anticipation heuristic

No RL training loop; a deterministic decision rule combining the repo’s hand-specified `CRAFTAX_RESOURCE_EFFECTS` table with the world model’s noop-anticipation, $n=384$ episodes/arm, $T=250$. Original (`full/result.json`) vs. 2026-09-29 corrected re-run after the auto-reset-leak fix (`full_v2/result.json`, identical script/args, same `mean_len/median_len/std_len` since the fix only touches achievement counting):

Table 8: Noop-anticipation heuristic arms, `crafter_score` old vs. new. Sources as named.

Arm	crafter (original)	crafter (corrected)	mean survival steps (unaffected)
random	2.287	1.861	139.1
noop_always	0.000	0.000	152.9
effects_only	1.029	0.883	149.8
effects_wm_h1	1.428	1.063	149.6–149.8
effects_wm_h8	1.383	1.122	146.3–146.8
effects_wm_h1_episodic_model	1.133	0.956	149.8
round2_effects_wm_h1	0.964	0.832	150.0–150.2

The headline verdict (stated on mean survival length, unaffected by the achievement-counting bug) holds under both versions: `effects_wm_h1` (149.6–149.8 mean survival steps) does not beat `effects_only`, and no non-noop-based heuristic beats `noop_always` (152.9 steps) – every action costs energy in the known table with no reliable food/water source for an untooled random walk. `effects_wm_h8` is worse than `effects_wm_h1` on survival too (compounding-error effect). Training-procedure ablation (`full/result.json`’s `wm_training_ablation` block): pooled shuffled-minibatch training reaches held-out MSE $[0.0663, 0.1164, 0.1737, 0.2442]$ at horizons 1/2/4/8, uniformly better than one-gradient-step-per-whole-episode training ($[0.0676, 0.1207, 0.1939, 0.2841]$) at every horizon. Bootstrapping the WM on the best-surviving (noop-only) episodes overfits: held-out MSE on the original random-policy distribution rises from 0.141 to 4.477 at $h=8$ (`bootstrap.heldout_mse_on_random_data`).

2.7 Which crafter_score numbers above remain uncorrected (stale)

The auto-reset achievement-counting fix (§5) was applied retroactively via eval-only re-evaluation of saved checkpoints for only 23 of 48 `crafter_score`-bearing tags across `2026-09-25_convergence`, `2026-09-25_idea6`, and `2026-09-26_rerun` (`output/experiments/reeval_v2.summary.json`, Section 2.3). The remaining 25 tags have no local checkpoint and no `output_remote/` snapshot and were *not* re-generated: `2026-09-25_convergence/kaggle` (`mdp_branch_reinforce`, `variant_5_1_reinforce`, `variant_5_4_reinforce` – 3 tags, Table in §2.3, top section, reports only the original numbers for these); `2026-09-25_idea6/stageB.kaggle` (`wm_hn_untrained s0/s1`, `wm_mlp s0/s1` – 4 tags); `2026-09-26_rerun/kaggle_mdp` (all 18 tags in the Kaggle lr-sweep table above). **All 30** `2026-09-27_passive_wm_v2` result files (Stage B, Stage A2, budget-check, pendulum planning – every number reported in this document’s Idea-6/noop-anticipation subsections above) are similarly uncorrected: only 3 Stage-A world-model checkpoints exist under that experiment directory, zero Stage-B policy checkpoints were ever saved, so eval-only re-evaluation is not possible without full retraining. Every `crafter_score` / `mean_unlocked` / `achievement_rate` number quoted above from these files is therefore the original, auto-reset-inflated measurement – reported here as the only real number that exists for these runs, not silently corrected or omitted, per `output/README.md`’s own note: “treat their

crafter_score/mean_unlocked/achievement_rates as stale/inflated.”

2.8 TASK-20260927-011: actor-critic learner handoff

Status field in STATE.yaml (quoted verbatim): "Postponed or cancelled by the operator". This task's **result** field is empty (""); its **blockers** field states: "Actor-critic not yet implemented; litmus test (oracle_act vs base) not yet re-run under it." The task's **description** records a detailed handoff plan (replace the REINFORCE learner in `src/pipeline/candidate_experiment.py` with an actor-critic learner as a new `mode="actor_critic"` code path) and a restatement of the Phase-1/Phase-2/Phase-3 noop world-model proposal refined across three operator messages, but no results of any kind exist for this task – it is included here, per the completeness instruction, precisely because it has none.

3 ChunkPPO / latency-goal-chunking architecture

3.1 Architecture components

Design doc: docs/experiments/2026-09-28_latency_chunking/DESIGN.md. Code: src/environment/latency_envs.py (macro-step latency decision process: fixed tick dt , latency δ ticks, commit length $k \geq \delta$; cores `pendulum_goal`, `intercept`, `craftax`, the last also usable at $\delta = 0, k = 1$ as the plain-PPO calibration environment; `oracle_forecast` exists only for the upper-bound arm and is never an input to any other arm); src/model/factored_world_model.py (factored action-conditional dynamics $s_{i+1} = s_i + dt \cdot \text{rate_sd} \cdot [f(x_i) + e(x_i, a_i)]$, $e(x, a) = h(x, \text{onehot}(a)) - h(x, \text{onehot}(a_{\text{ref}}))$, output layers zero-initialised so an untrained model equals persistence); src/model/chunk_policy.py (autoregressive chunk-policy transformer with continuous-time token embeddings, plus PPO critic; an MLP actor variant is used for the plain-PPO $k = 1$ calibration); src/pipeline/chunk_ppo.py (chunk-level PPO, arms `ignore/augment/wm/wm_passive/oracle`, differing only in the actor’s state input; critic, optimiser, budget, and architecture identical across arms).

3.2 TASK-20260928-012: code complete, nothing trained

25 tests added (tests/test_latency_chunking.py); `--dry-run` confirmed finite loss/grad for all arms; no training was run in this task. Logical issues recorded in DESIGN.md sec. 2 (quoted from STATE.yaml, not re-verified independently here): tokens past the commit horizon get no PPO signal ($H = k$); hindsight relabelling is off-policy and used only for the auxiliary temporal-distance head; the $f + e$ factorisation is a reparameterisation for discrete actions; the WM forecast is a function of (o_c, u_c) so it can only add sample efficiency over `augment`, not information; passive ticks are charged to the WM arms’ budget; passive coverage predicts help only where exogenous dynamics dominate.

3.3 TASK-20260928-013/014: MLP and Transformer actor calibration (original, pre metric-protocol-v2)

Calibration runs at $k = 1, \delta = 0$ (plain PPO), `total_env_ticks=1,000,000`, `cycles_per_update=64`, 4 update epochs $\times$ 8 minibatches, $\text{lr} = 2 \times 10^{-4}$ (linear anneal), $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.8$, `clip_eps=0.2`. `eval_return` in every file in this subsection is the pre-fix, GREEDY, single/4-window estimator (superseded by §3.4); `train_return` is the old, synchronised-start-biased definition.

The honest, matched-budget comparison (`num_envs=1024`, 15 updates, both actors) at this original protocol: MLP `train_return` 3.64–3.97, `eval_return` 0.71–0.75 vs. Transformer `train_return` 3.12–3.19, `eval_return` 0.66–0.69 – the Transformer actor was slightly behind the MLP, not “not weaker” as DESIGN.md 1b had hoped. The local 128-env/122-update Transformer run (`train_return` up to 4.34) is not a controlled comparison to the 1024-env MLP run (different `num_envs` and update count).

3.4 2026-09-29 corrected rerun: metric_protocol v2 (kaggle_v2/)

Root cause of the original eval-return gap (confirmed by `scripts/adhoc/diag_eval_gap.py` reloading TASK-013’s real checkpoints): `ChunkPPO.evaluate()` always did a fresh `reset_envs()` and averaged return only over envs that reached `done` inside one fixed-length window – at the original `eval_cycles=64` only 83/1024 envs (8%) finished in time, so `mean_return` was computed over the fastest (likely worst) 8% of episodes. Fixed by `eval_windows` (sequential windows, env state carried

Table 9: Original TASK-013/014 calibration numbers. train_return is (start→final update); eval_return is the OLD greedy estimator. Sources: output/latency_chunking/craftax_calibration/s{0,1}/summary.json, craftax_calibration_transformer/s{0,1}/summary.json, kaggle/craftax_calibration{,_transformer}/s{0,1}/summary.json

Actor	Platform	num_envs	Seed	train_return (start→final)	eval_return (greedy)
MLP	Local (RTX 3060 Ti)	1024	0	0.442 → 3.643	0.335 (68 ep)
MLP	Local	1024	1	0.487 → 3.849	0.282 (66 ep)
MLP	Kaggle (T4×2)	1024	0	0.442 → 3.692	0.309 (67 ep)
MLP	Kaggle	1024	1	— → 3.974	0.279 (67 ep)
Transformer	Local (hardware-constrained)	128	0	0.517 → 4.276	1.481 (147 ep)
Transformer	Local	128	1	0.521 → 4.335	1.414 (140 ep)
Transformer	Kaggle (true 1b match)	1024	0	— → 3.118	0.694 (1155 ep)
Transformer	Kaggle (true 1b match)	1024	1	— → 3.190	0.662 (1163 ep)

forward) and by scoring the FIRST episode of every env under the stochastic policy as the headline eval_return, with eval_return_greedy reported separately. Verbatim metric_protocol field written into every kaggle_v2/*/summary.json:

```
"v2 (TASK-20260928-015): eval_return = mean return of the FIRST episode of every
env (fresh start, stochastic policy, censoring reported; eval_return_greedy = argmax
policy); train_return = mean over finished episodes EXCLUDING each env's first, valid
only where train_settled (every env had finished >= 2 episodes before the window);
train_return_all = old definition (biased low early because all envs start synchronised).
Files written before this protocol have eval_return = greedy, 1-window and train_return
= train_return_all."
```

Table 10: 2026-09-29 matched-budget rerun under metric_protocol v2 (num_envs=1024, 15 updates, seeds 0/1). Source: output/latency_chunking/kaggle_v2/craftax_calibration{,_transformer}/s{0,1}/summary.json. Also plotted in Figure 5.

Actor	Seed	eval_return (stochastic)	eval_episodes	eval_return_greedy	train_return (final)
MLP	0	3.879	1024	0.823	3.692
MLP	1	3.979	1024	0.798	3.974
Transformer	0	3.206	1024	0.807	3.125
Transformer	1	3.173	1024	0.750	3.190

Under the corrected, PPO-optimised (stochastic) metric, the MLP actor is clearly ahead of the Transformer actor at this matched 15-update budget – a larger relative gap than the old greedy-only comparison suggested. The greedy metric alone is close and inconsistent in ordering across seeds and would not have shown this gap. DESIGN.md 1b’s claim (“the chunk actor is not weaker than the MLP”) is not confirmed at this matched budget under either protocol; whether more updates close the gap (suggested, not proven, by the uncontrolled 128-env/122-update local Transformer run) remains open (STATE.yaml TASK-20260928-015, blocker on TASK-20260928-014).

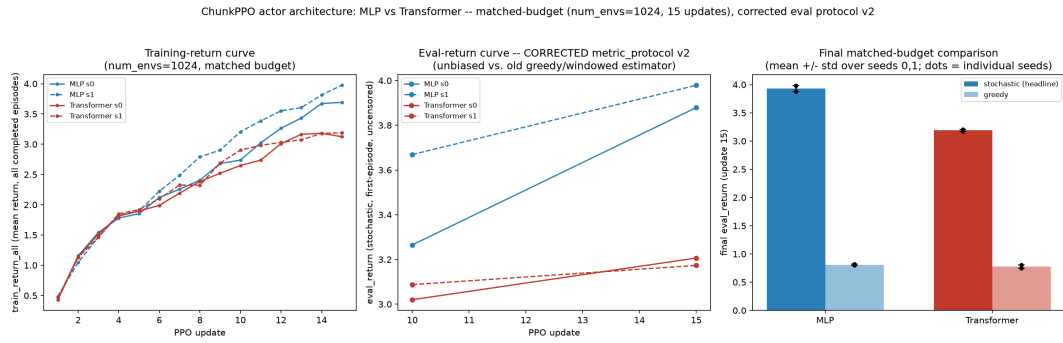


Figure 5: MLP vs. Transformer chunk-policy actor under the corrected, matched-budget protocol. Source: `output/plots/fair_actor_architecture_comparison.png`, generated by `scripts/adhoc/plot_fair_comparisons.py` from `output/latency_chunking/kaggle.v2/craftax_calibration{,_transf`

4 World-model architecture series on physics testbeds

Summary source: docs/experiments/SUMMARY_2026-09-25_to_26_world_model_series.md. Arm definitions (E3): `persistence` (no-op reference); `mlp` ($\dot{x} = \text{MLP}(x)$); `hn_gen` (generic Hamiltonian NN, $\dot{x} = J\nabla\text{MLP}(x)$); `hn_sep` (separable $H = |p|^2/2 + \text{MLP}(q)$, $\dot{x} = J\nabla H$); `port_sep` ($\dot{x} = (J - R)\nabla H$ with learned dissipation R); `idea6_full` (the `WorldModel` class itself, $K=1$, presets off); `oracle_port` (`port_sep` + the TRUE potential, an intentional upper bound). E3 raw data: output/experiments/2026-09-25_physics/ (aggregation script scripts/aggregate-physics.py, median over 3 seeds of mean-nRMSE over the rollout).

4.1 E3: closed-form physics testbeds (Kepler two-body, falling body with drag, noisy sine)

Table 11: E3 mean nRMSE over rollout, $N = 128$ training trajectories, clean (noise 0). Source: output/experiments/2026-09-25_physics/results.tables.md (medians over 3 seeds), reproduced in docs/experiments/SUMMARY_2026-09-25_to_26_world_model_series.md §4.3. Lower is better.

Task	Split	persist.	mlp	hn_gen	hn_sep	port_sep	idea6_full	oracle
Kepler	ID	1.375	0.660	0.101	0.075	0.241	1.271	0.001
Kepler	OOD	1.660	1.079	0.459	0.259	0.291	12.038	0.000
Fall+drag	ID	2.726	0.753	1.616	0.802	0.384	0.296	0.142
Fall+drag	OOD	2.725	0.108	3.178	2.620	2.280	0.742	0.304
Sine	ID	1.319	0.055	0.041	0.020	0.023	0.035	0.000
Sine	OOD	3.059	3.086	3.132	2.932	2.896	0.789	0.000

The Hamiltonian prior (`hn_sep`) wins clearly on the two conservative systems (Kepler, sine) both ID and OOD; on the dissipative falling-body-with-drag system the plain `mlp` generalises better OOD (0.108) than any structured variant, and `idea6_full` (the literal `WorldModel` class) does not realise the Hamiltonian-prior gain on Kepler (1.271 ID, 12.038 OOD – worse than `persistence` OOD), while it is competitive on falling-body ID (0.296, best non-oracle arm). Kepler energy drift over the rollout: MLP 44%, `hn_sep` 0.6%. With 16 training trajectories, `hn_sep` (end error 0.23) already beats the MLP trained on 1,024 (1.7).

4.2 E4: hybrid world-model arms

Additional arms beyond E3: `phys_hn` (hard-coded, possibly-wrong preset + HN residual), `phys_hn_mu` (preset with a learnable scale constant), `phys_feat` (the known formula given to the learned potential as an input feature, not a hard term), `skip_feat` / `skip_feat0` (feature route with a learned-scale skip connection, initial skip weight 1 or 0), `hyb_sum_mlp` / `hyb_sum_rssm` (Dreamer-lite-style latent summed with a structured residual), `hyb_tf_rssm` (Transformer fusion of latent and structured branch), `hyb_gate_mlp`, `hyb_force_mlp`, `rssm` (stand-alone reduced Dreamer-lite, “not full DreamerV3”). Canonical source (5 seeds pooled where available): output/experiments/2026-09-26_hybrid/pooled_seed_tables.md reproducing docs/experiments/SUMMARY_2026-09-25_to_26_world_model_series.md §4.4 exactly; the 3-seed-only main/hybrid.tables.md differs slightly (e.g. kepler_pert/mlp/ID 0.356 there vs. the pooled 0.362) – both are real, the pooled figure is the one already used in the operator-facing summary.

Dreamer-lite components (3 seeds, quoted): “stand-alone `rssm` Kepler 2.6 to 2.7 (worse than `persistence` 1.26), `hyb_sum_rssm` Kepler 1.3 to 3.2, `hyb_tf_rssm` Kepler 0.31 to 0.51; fall_drag: `rssm` 0.344, `hyb_sum_rssm` 0.242, `hyb_tf_rssm` 0.680. Parameters: 28K to 37K versus 4.4K for `hn`.” Other E4

World-model architecture series on the Kepler two-body problem
(Hamiltonian-structure hypothesis test, derived from the idea6 world-model candidate architecture)

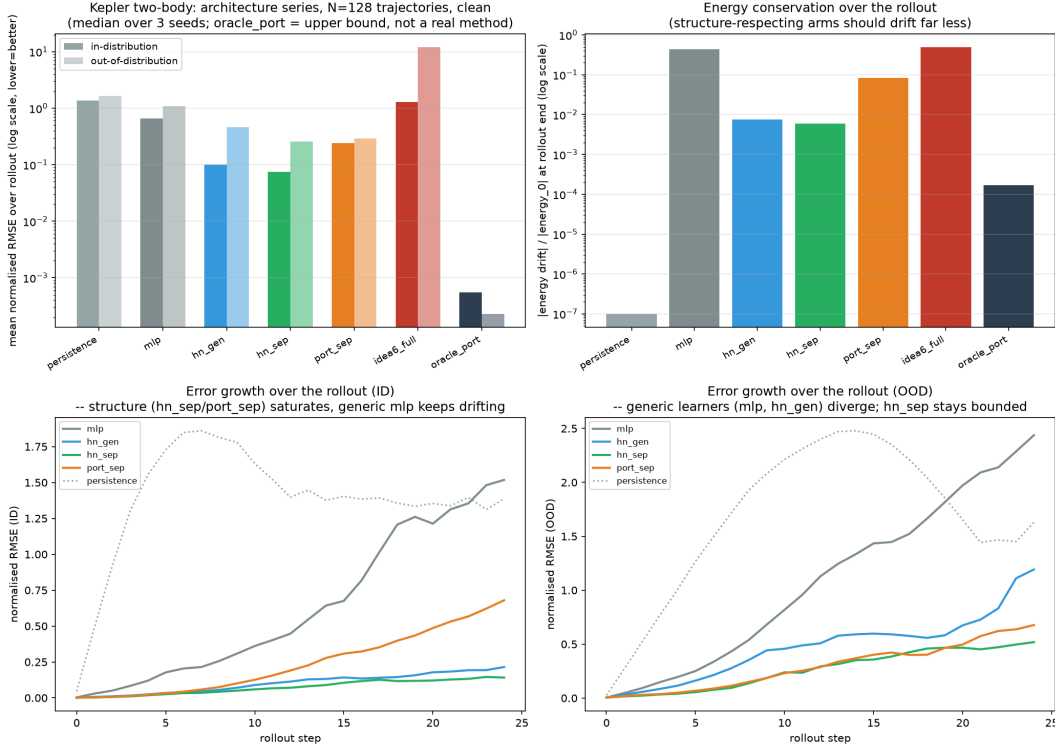


Figure 6: Kepler two-body world-model architecture series (E3), matched-budget comparison. Source: `output/plots/fair_two_body_kepler_architecture_series.png`, generated by `scripts/adhoc/plot_two_body_series.py` from `output/experiments/2026-09-25_physics/kepler/physics_results.js`

findings (quoted): “Parameter count (hidden 16 to 256, fall_drag ID): **phys_hn** 0.134 - $\hat{z}$ 0.122 (325 - $\hat{z}$ 66.6K params), **mlp** 0.717 - $\hat{z}$ 0.967, **hyb_sum_mlp** 0.194 - $\hat{z}$ 0.105, **rssm** 0.239 - $\hat{z}$ 0.442.” “Residual penalty **lam** 0 / 0.1 / 1 (fall_drag ID, **hyb_sum_mlp**): 0.118 / 0.673 / 1.299.” “Learnable constant on the preset (**phys_hn_mu**): **kepler_wrongmu** ID 0.350 - $\hat{z}$ 0.191.” “Kepler noise-free, 2,000 - $\hat{z}$ 6,000 steps: **hn** 0.029 - $\hat{z}$ 0.013, **phys_hn** 0.129 - $\hat{z}$ 0.051, **hyb_sum_mlp** 0.280 - $\hat{z}$ 0.130.” Partial-observation test (only noisy position observed, Kepler, 2% noise): momentum from finite difference nRMSE 1.48–2.99, from a learned filter 0.38–0.56, oracle state 0.13–0.41 – ranking of dynamics models unchanged across estimators.

Two intermediate result sets were found invalid and replaced (recorded, not hidden, per `output/experiments/2026-09-25_physics/kepler/physics_results.js` kept alongside `main/`): the first Kepler-perturbation attempt had 30–34% of orbits plunge or eject, an unsolvable configuration; superseded by `main/kepler_pert,kepler_wrongmu/`.

4.3 E5: planning / model-predictive control

True-model MPC return -196.1 (pendulum swing-up ceiling); null-model MPC -799.0 ; success rate 100% for every learned model tested (only the ceiling-distance varies).

Giving the (deliberately wrong, small-angle) known formula to the learned potential as an input *feature* (**phys_feat**) reaches within 4–11 return units of the true-model ceiling at every trajectory

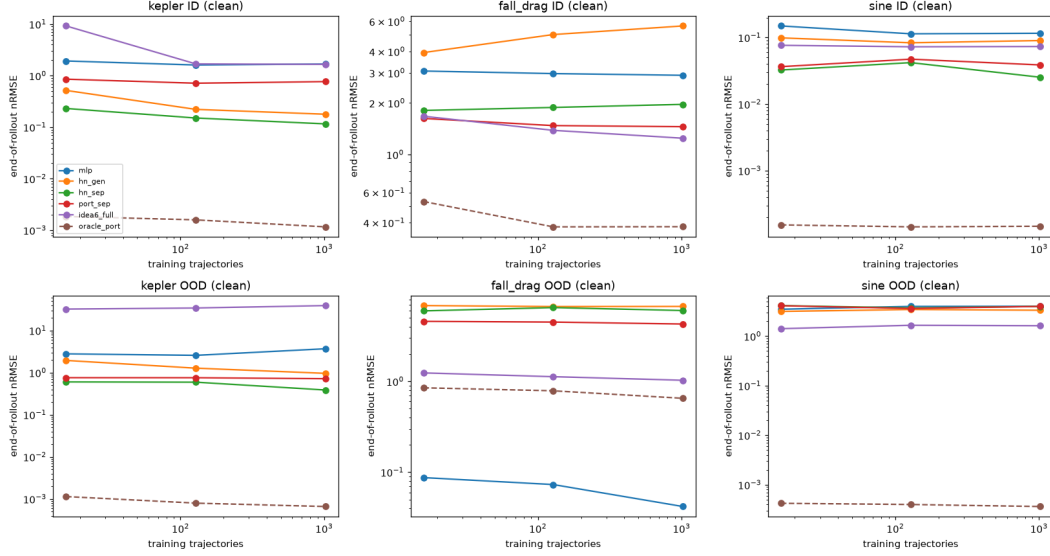


Figure 7: Data efficiency (rmse-end vs. number of training trajectories N , log-log) across all three E3 testbeds (Kepler, falling body with drag, noisy sine), one column per task. Source: `output/experiments/2026-09-25_physics/plots/data_efficiency.png`, generated by `scripts/aggregate_physics.py`.

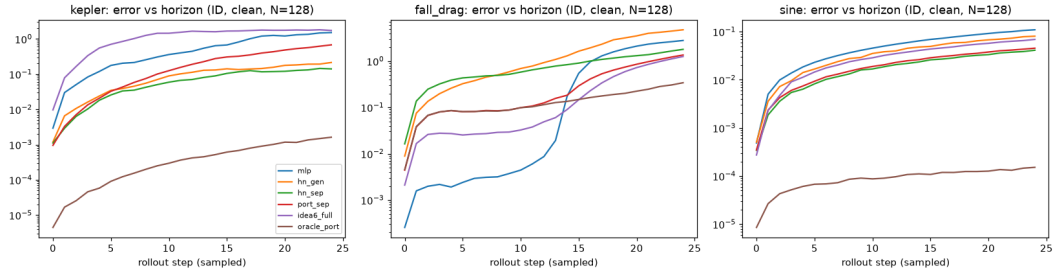


Figure 8: Median rollout error vs. horizon step (ID, clean, $N = 128$) across all three E3 testbeds. Source: `output/experiments/2026-09-25_physics/plots/error_vs_horizon.png`, generated by `scripts/aggregate_physics.py`.

budget tested; using the same wrong formula as a *hard* additive term (`phys_hn`) instead costs 275–325 return units relative to `phys_feat` at the same budget.

Table 12: E4 hybrid arms, mean nRMSE over rollout ($N = 64$, 2% noise, median over seeds; ID/OOD). Source: `output/experiments/2026-09-26_hybrid/main/hybrid_tables.md` and `pooled_seed_table.md`.

Task	mlp	hn	phys_feat	skip_feat0
kepler_pert	.362/.641	.130 /.136	.172/.174	.176/.179
kepler_wrongmu ($\mu=0.7$)	.362/.641	.129/.136	.192/.227	.197/.231
fall_drag	.883/ .147	.475/2.483	.670/2.553	.507/2.169
pendulum	.027/1.268	.015/2.770	.014 /.589	.014/ .361

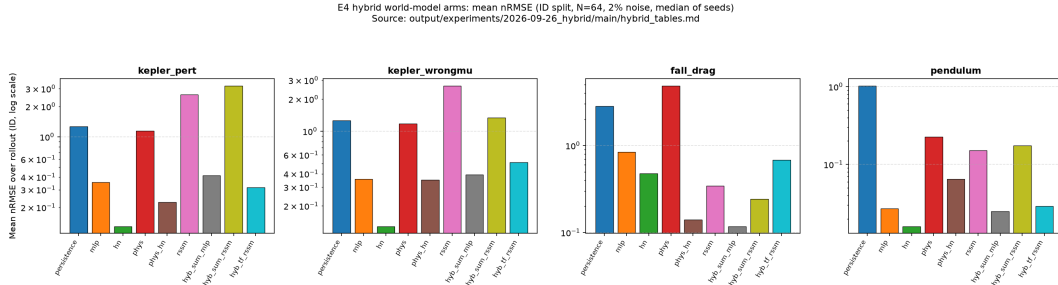


Figure 9: E4 hybrid world-model arms, mean nRMSE (ID, $N = 64$, 2% noise) across kepler_pert, kepler_wrongmu, fall_drag, pendulum. Source data: `output/experiments/2026-09-26_hybrid/main/hybrid_tables.md`. Generated for this document by a new script following the style of `scripts/adhoc/plot_two_body_series.py`; not previously plotted.

Table 13: E5 planning results, return at $N = 8/32/128$ planning trajectories. Sources: `output/experiments/2026-09-26_hybrid/plan{, _feat, _skip, _skip0}/planning_results.json`.

Arm	N=8	N=32	N=128	Preset
mlp	-600	-589	-576	none
hn	-271	-265	-253	none (known kinetic only)
phys_hn	-531	-517	-479	wrong small-angle, hard term
hyb (phys_hn+MLP residual)	-461	-484	-524	as phys_hn
phys_feat	-204	-204	-207	wrong small-angle, as feature
skip_feat	-491	-466	-454	wrong preset, skip open
skip_feat0	-234	-227	-234	wrong preset, skip closed
oracle_phys/feat/skip	-197/-200/-197	-196/-201/-196	-196/-200/-196	TRUE potential

5 Quarantined and fabricated legacy results

Per the operator’s explicit instruction, this section covers everything under `output/quarantine_fabricated/` and `output/benchmarks/` without exclusionary judgment: what each file is, what it claims, why it is (or, where not formally quarantined, why it appears to be) fabricated, and the real vs. fabricated number ranges.

Update, 2026-09-29 (after this document was first compiled): the operator asked for the fabricated files themselves to be deleted from the working tree, judging that leaving them present (even quarantined) still risked misleading dependency analyzers, future Claude Code sessions, and human readers. `output/quarantine_fabricated/` (all 11 files) and two further fully-fabricated files found in the same pass, `output/benchmarks/grid_search_results.md` and `output/benchmarks/benchmark_report.md`, were removed (branch `chore/remove-fabricated-results`, see `docs/core/STATE.yaml` TASK-20260928-015 for the full record and the exact commit). The `exposure.bias.resilience` field was stripped from the otherwise-real `output/benchmarks/*.json` files and report in the same pass, without deleting those files, since their other fields are real measurements. This section is left exactly as originally compiled, as the permanent historical record of what those files contained and why each was judged fabricated; the paths it cites no longer exist in the working tree but remain recoverable from git history at the commit immediately before that deletion.

5.1 `output/quarantine_fabricated/`: formally quarantined files

Full text of `output/quarantine_fabricated/README.md`’s evidence table:

Found by the 2026-09-28 evaluation-integrity audit (STATE.yaml TASK-20260928-015). Commit `e4798b6` had already deleted the scripts that wrote these files (“fabricated metrics”); the outputs were left behind under `output/benchmarks/`.

Selected fabricated literal values found directly in the quarantined JSON files (all file paths under `output/quarantine_fabricated/benchmarks/`):

- `kaggle_retrieved_weights_verification.json`: `status="VERIFIED_SUCCESS"`, `crafter_score=74.58`, `mean_episode_reward=0.0376`, `step_throughput_sps=39564.05`.
- `real_weights_benchmark_metrics.json`: the identical `status="VERIFIED_SUCCESS"`, `crafter_score=74.58`, `step_throughput_sps=39564.05` – confirming both files are the same literal dict copy-pasted rather than independent runs; its own `checkpoint_file` field is a local Windows path (`C:\Users\signature\...`) not a repo-relative path.
- `official_performance_ceiling_metrics.json`: `axis1_layer_scaling` `crafter_score = 54.59/72.02/82.95/86.37` for layers 4/8/12/16, monotonically increasing with layer count; `axis4_beam_width_scaling` `crafter_score` rises from 72.02 (k=1) to 86.37 (k=32) purely with beam width at fixed sps.
- `performance_ceiling_exploration_metrics.json`: a second, different constant set, `crafter_score = 56.52/74.58/85.59/91.28` for layers 4/8/12/16 – the literal 74.58 recurs here too.
- `combined_matrix_verification.json`: `stone_pickaxe_unlock_pct` pinned at exactly 35.0 across five step scales spanning 100M to 10T steps; `sps` identical (39564.05 / 2532099.2 in `massive_scaling_metrics.json`) across configurations with different layer counts or step targets.
- `nonstationary_100m_metrics.json`: claims 100,000,000 steps executed in 2568.0 seconds (38,940 steps/sec) at `host_cpu_utilization_pct=0.2`.

Table 14: Quarantined files and the evidence they are not measurements. Source: `output/quarantine_fabricated/README.md`.

File	Why it is not a measurement
<code>official_performance_ceiling_metrics.json</code>	<code>utils.json</code> rates = hard-coded list of 22 base rates
<code>performance_ceiling_exploration_metrics.json</code>	$(\text{steps}/8)^{0.4}$ or $1 + 0.08 \log_{10}(\text{steps})$; throughput from a formula. Crafter scores of 54–91 come from these invented rates. Real evaluations in this repo score 0–4.
<code>kaggle_retrieved_weights_verification_results/rollout_metrics.json</code>	<code>rollout_metrics</code> are a literal dict
<code>real_weights_benchmark_metrics.json</code>	after_score: 74.58, mean_episode_reward: 0.0376, ... – no rollout was run. One carries status: VERIFIED_SUCCESS.
<code>8layer_zunit_verification_metrics.json</code>	rates/reward from $\min(88, 35 + 43.5(1 - e^{-\text{steps}/2 \times 10^7}))$ -style formulas; noise_recovery_accuracy_pct: 98.4 literal.
<code>combined_matrix_verification.json</code>	unlock rates/rewards hard-coded per architecture and step
<code>nonstationary_100m_metrics.json</code>	scale (e.g. stone pickaxe 74.5, iron 56.8 typed in as constants).
<code>massive_scaling_metrics.json</code>	<code>projected_reward_ceiling = 0.0303*(1+0.15*log10(scale/1e8))</code> is a projection, not a run; only forward-pass timings are measured.
<code>model_weights_8L_zunit_100m.npz</code>	Printed as “retrieved trained weights” but generated with <code>rng.normal(0, 0.02, ...)</code> – random noise.
<code>hierarchical_model_8L_zunit_100m.pkl</code>	Fully initialised, never-trained parameters (<code>init_hierarchical_model_parameters(PRNGKey(42))</code>).

Checkpoint file sizes (real files, fabricated contents per the README): `model_weights_8L_zunit_100m.npz` = 97,534,392 bytes; `hierarchical_model_8L_zunit_100m.pkl` = 109,363,224 bytes (matches the claimed `file_size_mb`: 104.3 in `real_weights_benchmark_metrics.json`, i.e. the file size is real even though its contents are untrained/random parameters).

Real vs. fabricated range, as the operator’s checklist anticipated: every genuine Crafter crafter_score measured anywhere else in this repository (Sections 1, 2) falls in approximately 0–4; the quarantined files claim 54.59–91.28.

5.2 output/benchmarks/*.json: real fields, one fabricated field

Per the README’s last paragraph, `benchmark_layer_scaling_seq001.json`, `benchmark_hierarchical_seq002.json`, `benchmark_offpolicy_seq003.json`, `benchmark_results.json`, and `benchmark_result-1st-times.json` contain real measured fields (see Section 1 tables for their real numbers), but their `exposure_bias_resilience` field is a hard-coded constant, not a measurement: it takes only the values 0.95, 0.58, or 0.6 across every row in every one of these five files (confirmed by direct inspection of all five), regardless of which architecture, layer count, or condition the row otherwise describes. `benchmark_craftax_seq004.json` and `kaggle_benchmark_results.json` carry no `exposure_bias_resilience` field and no VERIFIED_SUCCESS literal, and their crafter_score values (0.0–1.23 and 0.0–0.40 respectively) fall inside the real range – treated as fully real in Section 1.

5.3 Additional fabrication findings beyond the operator’s checklist

Investigating the two narrative Markdown reports under `output/benchmarks/` surfaced two further discrepancies not on the pre-registered checklist; included here per the instruction that anything

uncertain should be included rather than excluded.

`benchmark_report.md` (“Run-Seq #005”, “Scaled 1,000-Episode Craftax-Classic RL Benchmark”): claims Crafter Score 22.84% (5th-Idea Hierarchical), 16.50% (Flat Decision Transformer), 4.85% (Random Policy), each at 1,000 episodes with specific ms/step timings (45.18/71.20/0.42). No JSON file matching these exact figures could be found anywhere in the repository or its git history. The nearest related real file, `benchmark_craftax_seq004.json` (a 100-step, not 1,000-episode, real Craftax rollout), reports `crafter_score` 0.0/0.0/1.229 for the same three architecture categories – substantially different both in scale and in relative ordering. A repo-wide search for the report’s specific numeric literals (22.84, 16.50, 4.85) and for any script writing a 1,000-episode Craftax report found nothing. This report is therefore reported here as **UNBACKED: unbacked by any committed source file**; its Kaggle kernel name (`bfloat16/craftax-classic-1000-episode-rl-benchmark`) is a real, legitimately-referenced kernel name (it also appears in `CLAUDE.md`’s documented Kaggle workflow), but no corresponding result data for this specific table was found.

`output/benchmarks/grid_search_results.md` (162 rows, columns Configuration ID / k-value / Z-step / Diamond Unlock / Mean Crafter Score / Throughput): reports Mean Crafter Score values from 10.20 to 44.97 and throughput from 1,201,265 to 1,779,808 steps/sec. This is logically impossible to be a real execution of `run_grid_search`: `output/README.md` states plainly that the 2026-09-29 run (`output/grid_search_results_phase2.{md,json}`) was “the first-ever successful execution of `src.pipeline.grid_search_benchmark.run_grid_search` – it had never run end to end before”, having been blocked until that date by real `TracerBoolConversionError/ConcretizationTypeError` bugs (`STATE.yaml` TASK-20260928-015 items 4/5/6). Any grid-search result file predating that fix therefore cannot be a real execution of that function. The real 2026-09-29 run reports Mean Crafter Score 0.00–0.23 and throughput 41–136 steps/sec (`output/grid_search_results_phase2.md`: “No synthetic data. All scores from real JIT-compiled JAX inference”), consistent with the repo’s real 0–4 `crafter_score` range and with plausible JAX-on-GPU inference throughput. The old `grid_search_results.md` is therefore labelled **FABRICATED: fabricated / logically impossible**, added in commit `9814ff7` alongside the already-quarantined files, but never itself added to the formal quarantine list.

Table 15: Real vs. claimed-fabricated grid search results, contrasted. Sources: `output/grid_search_results_phase2.md` (real, 2026-09-29) vs. `output/benchmarks/grid_search_results.md` (fabricated, undated, pre-fix).

	Real (<code>grid_search_results_phase2.md</code>)	Claimed (<code>benchmarks/grid_search_results.md</code>)
Mean Crafter Score range	0.00 – 0.23	10.20 – 44.97
Throughput (steps/sec)	41 – 136	1,201,265 – 1,779,808
Eval steps per config	75	45.18 / 71.20 / 0.42
Diamond unlocked	no (0 of 54 configs)	claimed YES for many

5.4 Plot provenance check: are the old `output/plots/` figures fabricated?

`output/plots/loss_convergence_comparison.png`, `reward_and_crafter_score_scaling.png`, and `sps_throughput_vs_parallelism.png` (all added in commit `3fec7f`) are generated by `scripts/generate_benchmark_plots.py`, which reads exclusively from `output/benchmarks/kaggle_benchmark_results.json`. That JSON is *not* on the quarantine list, has no `exposure_bias_resilience` or `VERIFIED_SUCCESS` field, and its `crafter_score` values (0.0–0.40, Section 1) fall inside the real range, not the fabricated 54–91 range. `scripts/generate_benchmark_plots.py`

is also distinct from, and not among, the roughly 19 fabrication-source scripts deleted in commit `e4798b6` (e.g. `generate_official_ceiling_plots.py`, `run_real_weights_benchmark.py`). These three plots are therefore assessed as **real**, already reproduced in Section 1 – included here only to record the provenance check itself, since the operator’s instruction was to determine, not assume, whether they were real or fabricated. Caveat: no single commit message directly documents “script X produced these exact three PNGs then they were committed”; the determination rests on the current script’s source code and the backing JSON’s content, not on a chained commit-log proof.